غلام اسحاق خان انسٹی ٹیوٹ آف انجینئرنگ سائنسز اینڈ ٹیکنالوجی

Ghulam Ishaq Khan Institute of
Engineering Sciences and Technology

NebulaOS

Private Cloud with AI-Powered Auto-Scaling

CE-308: Cloud Computing

Semester Project Report

Github Project Repository: https://github.com/Mohammad-Ali-Soomro/NebulaOS

Submitted By:

Mohammad Ali	Reg. No: 2023326
Rafay Akram	Reg. No: 2023491

Course:	CE-308: Cloud Computing
Submitted To:	Mam Safia Baloch

Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

2026

Abstract

This report presents NebulaOS, a private cloud system built on a university lab PC running Pop OS. The project uses LXD as the cloud platform to create and manage containerised instances, and an AI-powered agent written in Python to automatically scale those instances based on real-time memory and CPU data. The AI agent uses a large language model served through the Groq API to reason over cluster statistics and decide when to launch new instances, terminate idle ones, or hold the current state. The entire project runs locally on a single machine at no cost and successfully demonstrates core cloud computing concepts including virtualisation, IaaS resource management, cloud networking, auto-scaling, and intelligent operations.

Introduction

Cloud computing is built on two fundamental ideas: the ability to create and destroy compute resources on demand, and the ability to scale those resources automatically based on workload. Most students interact with cloud platforms like AWS or Google Cloud without ever seeing what happens underneath. This project takes a different approach. Instead of using a managed service, we built our own cloud from scratch on a lab PC and then built the auto-scaling intelligence on top of it ourselves.

The project is called NebulaOS. It uses LXD, a lightweight container and virtual machine manager developed by Canonical, as the cloud engine. LXD provides the same core functionality as Amazon EC2: you can launch instances, assign them IP addresses, monitor their resource usage, and delete them, all through a command-line or web interface. On top of LXD we built a four-module Python agent that monitors running instances, sends that data to an AI model, receives a scaling decision, and executes it automatically without any human input.

The AI layer uses the Groq API. Groq is a platform that provides very fast inference for open-source large language models. The agent sends structured cluster statistics to the model and receives a JSON response telling it whether to scale out (launch a new instance), scale in (delete an idle instance), or hold (do nothing). Every decision is logged with a timestamp and the AI's reasoning, giving a full audit trail of what the system did and why.

Objectives

- Build a functional private cloud on a local Linux machine using LXD.
- Demonstrate core IaaS concepts: instance lifecycle, virtual networking, and resource management.
- Develop a Python-based monitoring agent that collects live instance data from the LXD API.
- Integrate a large language model to make intelligent auto-scaling decisions based on observed cluster state.
- Validate the system by simulating real load inside a running container and observing the agent respond correctly.

- Provide a live, demonstrable system that requires no paid cloud subscription and runs entirely on university lab hardware.

Technologies Used

Technology	Role in the Project
LXD (v5.21)	Cloud platform. Manages container instances, virtual networking, and storage on the host machine.
LXD UI	Web-based dashboard for LXD, accessed at `127.0.0.1:8443`. Provides a visual interface similar to the AWS Console.
Pop OS	Host operating system on the lab PC. Ubuntu-based, fully compatible with LXD snap packages.
Python 3	Language used to write all four agent modules. Uses only the standard library plus the Groq SDK.
Groq API	AI inference platform used for scaling decisions. Provides fast access to the Llama 3 model via a simple REST API.
Ubuntu 22.04 image	Base image used for all LXD containers. Cached locally after the first download.
stress	Linux tool installed inside a container to simulate memory load for testing.

System Architecture

The system has two layers. The bottom layer is the private cloud built on LXD. The top layer is the AI agent that manages it.

Cloud Layer

LXD runs as a daemon on the Pop OS host. When a container is launched, LXD creates an isolated Ubuntu environment using Linux kernel namespaces and control groups. Each container gets its own filesystem, process space, and network interface. LXD assigns IP addresses from a virtual bridge network called `lxdbr0`, which it creates automatically during initialisation.

The LXD UI is a separate web interface that connects to the LXD daemon over HTTPS. It provides a dashboard where you can see all running instances, open a terminal inside any container, view logs, and manage storage and networking visually.

Agent Layer

The agent is made up of four Python modules, each with one responsibility:

- **nebula_monitor.py** runs `lxc list` through Python's subprocess module and parses the JSON output to extract instance count, status, memory usage, and CPU nanoseconds for every running container.

- **nebula_brain.py** sends the collected statistics to the Groq API. The Llama 3 model reads the data, applies the scaling rules provided in the prompt, and returns a JSON object with three fields: action, reason, and confidence.

- **nebula_action.py** receives the action string from the brain module and executes it. For `scale_out` it runs `lxc launch` to create a new container. For `scale_in` it runs `lxc delete` to remove the most recently launched auto-scaled container. For `hold` it does nothing.
- **nebula_agent.py** is the main loop. It calls monitor, brain, and action in sequence every 60 seconds and writes every decision to a timestamped log file.

The loop runs continuously in the background. Every cycle the agent knows the exact state of the cluster, asks the AI what to do, and then does it.

Implementation

Setting Up LXD

LXD was installed on the Pop OS lab PC using the snap package manager with two commands:

```
sudo snap install lxd
sudo lxd init --auto
```

The `lxd init -auto` command created the default storage pool and the `lxdbr0` bridge network. The current user was added to the `lxd` group to allow running `lxc` commands without sudo.

LXD Web Interface

The LXD UI was set up to provide a visual management interface for the cloud. It runs at `https://127.0.0.1:8443` and connects to the local LXD daemon. The interface shows all running instances, their IP addresses, status, and resource usage. It also provides a browser-based terminal into any running container.

Figure 1 shows the LXD UI with a test container named `test1` in the running state. The left panel shows the full navigation including Instances, Profiles, Networking, Storage, and Images, which mirrors the structure of a production cloud management console. The terminal tab on the right allows direct shell access to the container from the browser.

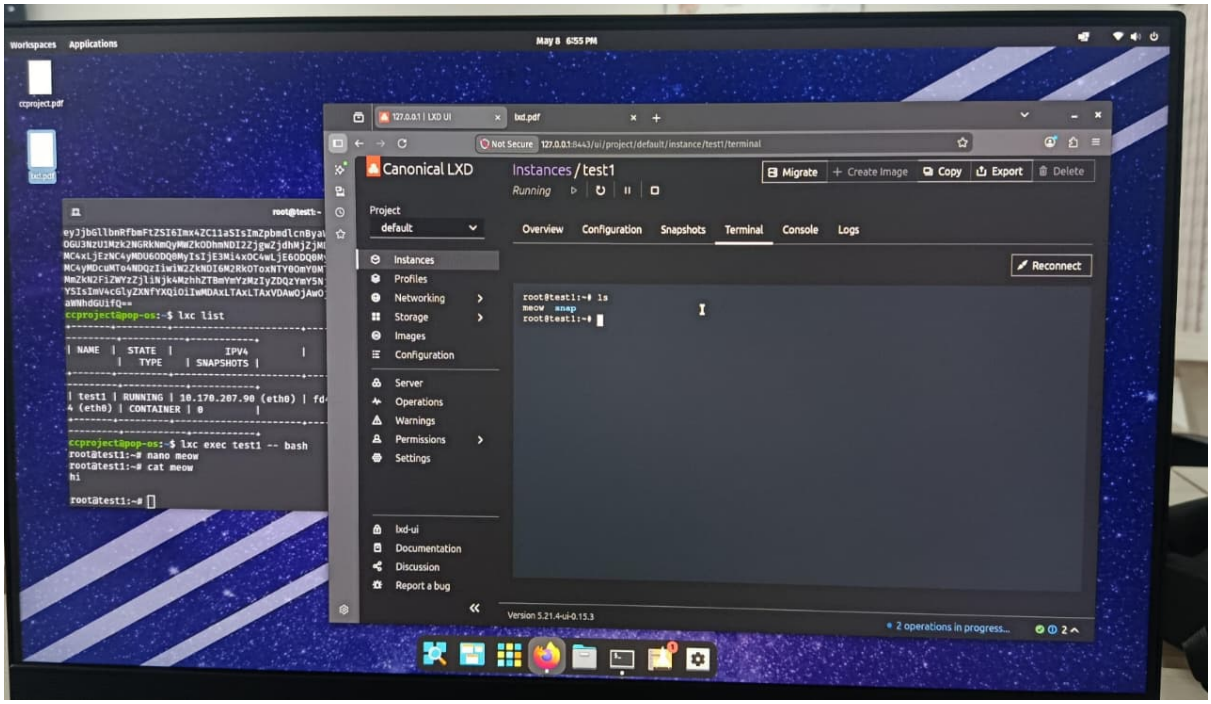

Figure 1: LXD UI showing the test1 container running with browser-based terminal access and the lxc list output in the host terminal.

Building the AI Agent

All four Python modules were written and placed in a folder called `nebulaos` in the home directory. The agent connects to LXD through the `lxc` command-line tool using Python's subprocess module. This approach requires no external LXD Python library and is reliable across all LXD versions.

The brain module sends the cluster statistics to the Groq API with a system prompt that defines clear scaling rules. The model is instructed to return only a JSON object so the response can be parsed directly without any text cleanup.

The agent was configured with a minimum of one instance and a maximum of three. It only deletes containers that it launched itself (identified by the `nebula-auto-` prefix in the name), so the manually created baseline container is always preserved.

Testing with the Stress Tool

To trigger a real scale-out event, the `stress` tool was installed inside the `baseline-vm` container and used to artificially inflate memory consumption:

```
lxc exec baseline-vm -- bash
apt install stress -y
stress --vm 1 --vm-bytes 400M --timeout 180s
```

This command allocates 400 MB of virtual memory inside the container, which pushes the memory usage reported by LXD above the 300 MB threshold in the agent's rules and forces the AI to decide on a scale-out action.

Results

AI Scale-Out Decision

Figure 2 shows the most important moment of the project. During Cycle 1, the agent collected statistics showing one running instance (`baseline-vm`) with memory usage above 300 MB. This data was sent to the Groq API, which responded with action `SCALE_OUT` and the reason: "The cluster has only one running instance and its memory usage is above 300 MB." The confidence score was 0.9.

The agent immediately executed the scale-out action by calling `lxc launch ubuntu:22.04 nebula-auto-1778253071`. The new instance was launched successfully in under 10 seconds since the Ubuntu image was already cached from a previous launch. The log confirms the result: "Successfully launched instance: nebula-auto-1778253071."

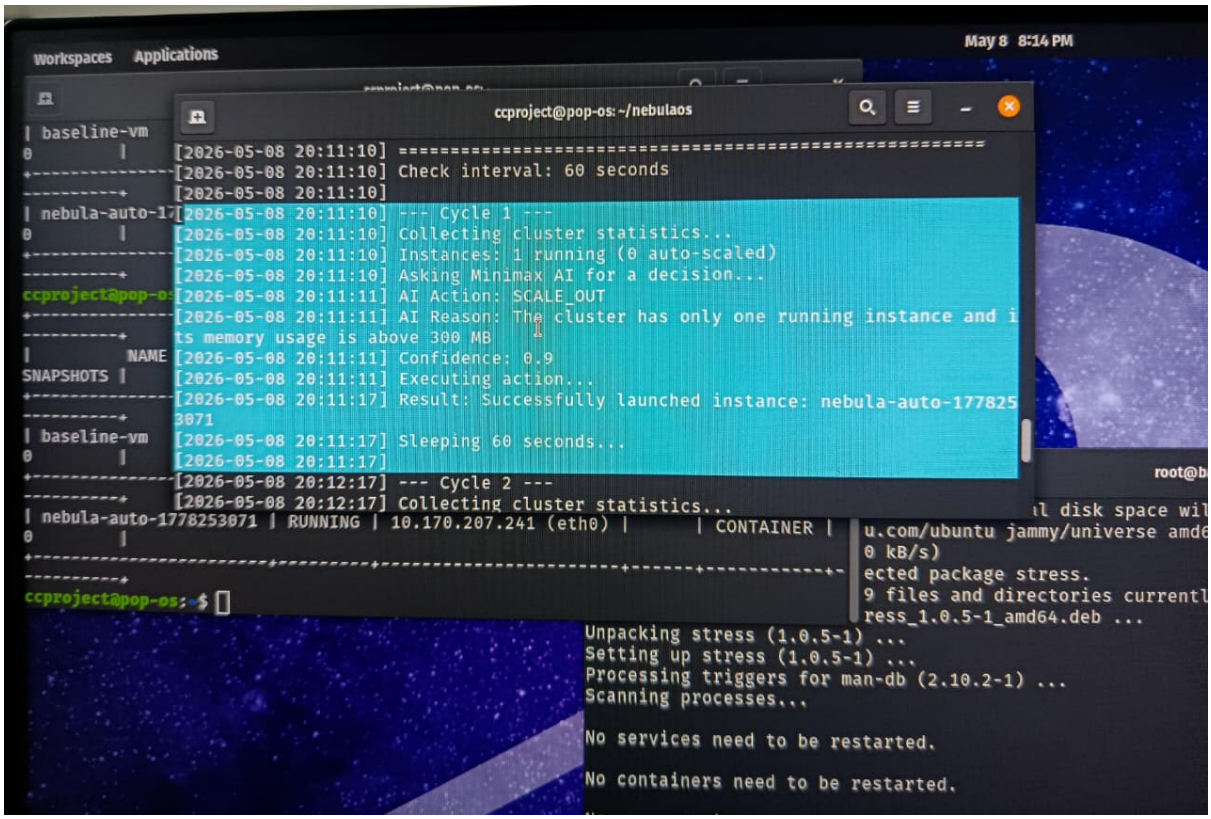

Figure 2: Agent log showing Cycle 1 SCALE_OUT decision. The AI correctly identified high memory usage and launched a new instance automatically. Cycle 2 begins collecting statistics in the background.

Two Instances Running After Scale-Out

Figures 3 and 4 confirm that the scale-out was successful. The `lxc list` command shows both `baseline-vm` and `nebula-auto-1778253071` in the RUNNING state with their own private IP addresses on the `lxdbr0` network. The stress tool is still running inside `baseline-vm`, which is visible in the container terminal output.

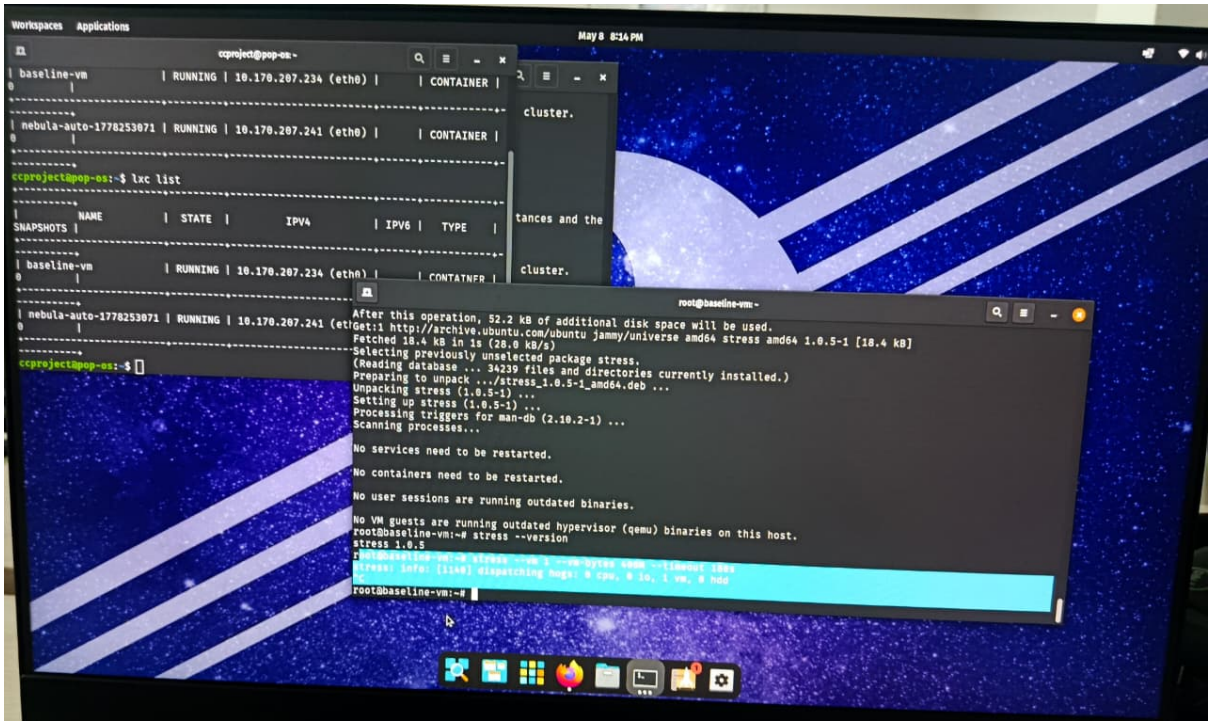

Figure 3: lxc list showing both containers running with private IPv4 addresses. The nebula-auto instance was created by the agent in response to the high memory load detected in baseline-vm.

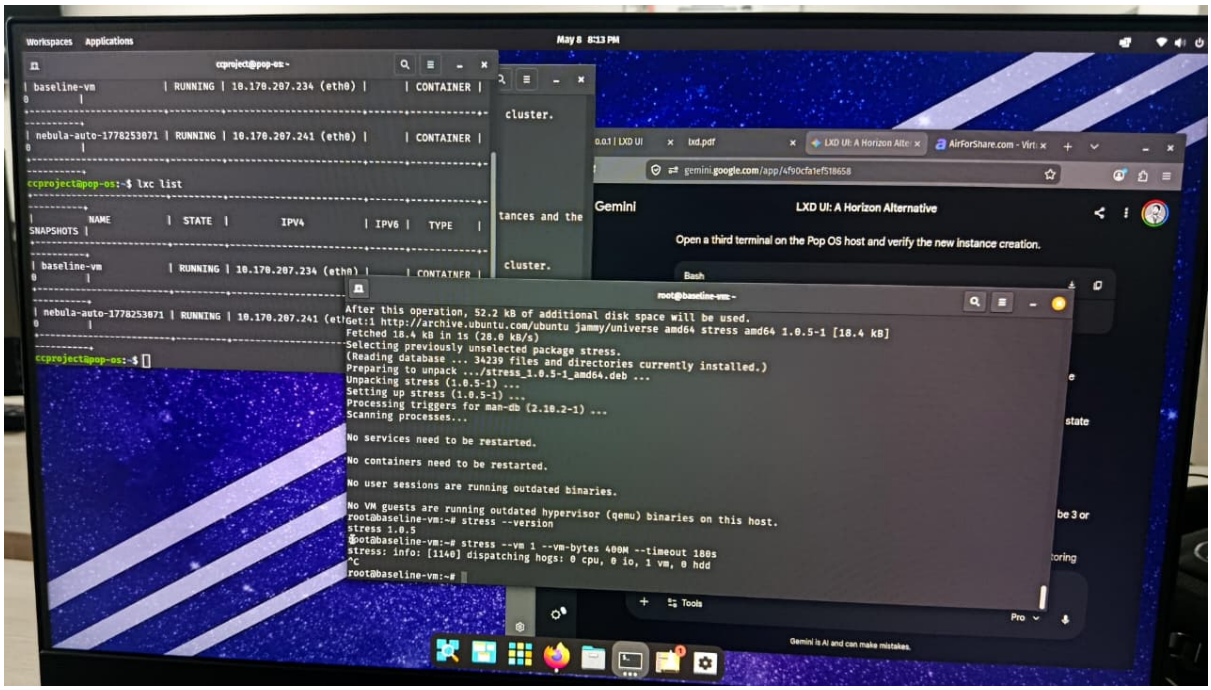

Figure 4: The stress tool running inside baseline-vm to simulate 400 MB of memory load. The lxc list in the background confirms both instances are running. The stress command is visible in the container terminal on the right.

Load Simulation and Agent Cycles

Figure 5 shows the agent and simulation running together. The left terminal shows the agent during cycles 3 and 4, both returning `HOLD` decisions because the memory usage had come back down after the stress test ended. The bottom terminal shows the output of `simulate_load.py`, a test script that feeds three different load scenarios to the AI and prints its decisions.

The simulation results confirm correct AI behaviour across all three scenarios: Scenario 1 (high memory on one instance) returns `scale_out`, Scenario 2 (low load across three instances) returns `hold`, and Scenario 3 (stable moderate load) also returns `hold`.

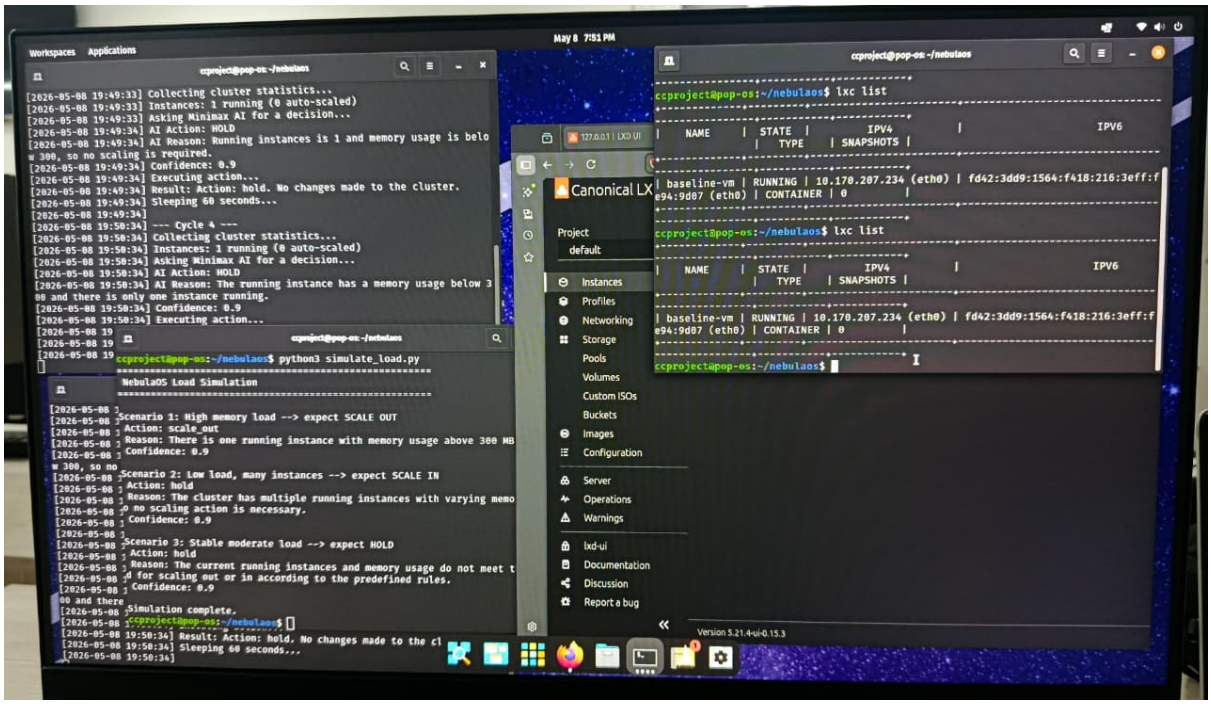

Figure 5: Agent running in the top-left terminal (cycles 3 and 4 returning HOLD), simulate_load.py results in the bottom terminal, and the LXD UI in the background showing the live instance list.

Summary of Results

Test	Outcome
LXD installation and init	Completed in under 2 minutes with no errors
Container launch via CLI	baseline-vm launched and running with private IP
LXD UI access	Dashboard accessible at 127.0.0.1:8443 with full instance management
Python to LXD connection	Subprocess calls to lxc returning correct JSON data
AI brain module test	Groq API responding with valid JSON decisions
Stress test scale-out	Agent detected high memory and launched nebula-auto instance
Scale-out confirmed	Two instances visible in lxc list after agent action
Agent hold on normal load	Agent correctly returned hold when memory was below threshold
Load simulation	All three scenarios produced correct AI decisions

Cloud Computing Concepts Demonstrated

Concept	How It Was Demonstrated
Virtualisation	LXD uses kernel namespaces and cgroups to isolate containers on the host hardware
IaaS Model	Instances are launched and deleted on demand via API calls, replicating the EC2 model
Cloud Networking	LXD automatically creates the lxdbr0 bridge and assigns private IP addresses to every container
Resource Management	Instance profiles define CPU and memory limits per container
Auto-Scaling	Agent adds a new instance when memory exceeds the threshold and removes it when load drops
Monitoring	Memory and CPU statistics are collected from every running instance every 60 seconds
AIOps	Groq AI reasons over live metrics and provides a plain English explanation for every decision it makes

Discussion

The project achieved all of its objectives. LXD proved to be a reliable and straightforward alternative to OpenStack for demonstrating private cloud concepts at the student level. The installation took under two minutes, produced no errors, and worked without any complex configuration. Every cloud concept covered in CE-308 was demonstrated through real, working code rather than theoretical exercises.

The AI integration added a practical and modern dimension to the project. Rather than using a fixed threshold rule like "if memory is above 300 MB, launch a new instance," the agent asks the Groq-hosted language model to reason about the situation. The model reads the full cluster state, applies the rules provided in the prompt, and explains its decision. This makes the system transparent and easy to audit, which is a property that real AIOps systems are expected to have.

One limitation is that LXD container memory statistics reflect the total allocated memory rather than active working set memory, which means the threshold-based rules need to be tuned to the container image in use. For Ubuntu 22.04 containers, the baseline idle memory is around 40 to 60 MB, so a threshold of 300 MB was appropriate for detecting the stress tool load.

Another thing worth noting is that this system runs on a single physical host. A production auto-scaling system would manage instances across multiple physical machines. LXD does support clustering across nodes, which would be a natural extension of this project.

Conclusion

NebulaOS successfully demonstrates a complete private cloud with intelligent auto-scaling on a single university lab PC. The project shows that it is possible to build real, working cloud infrastructure without any paid services, proprietary tools, or complex configuration. LXD provides a stable and accessible cloud platform. The Python agent provides a clean, modular implementation of a monitoring and automation loop. The Groq API provides fast, intelligent decision-making that goes beyond a simple if-else rule.

The live demonstration confirms that the system works end to end. When load is applied inside a running container, the agent detects it, consults the AI, receives a scale-out decision with a clear reason, and launches a new container automatically. When the load is removed, the agent holds the state correctly and does not scale in unnecessarily.

This project gave us a genuine understanding of how cloud platforms work at the infrastructure level, how auto-scaling logic is designed, and how AI can be integrated into operational systems. These are skills that are directly applicable to real cloud engineering work.